CLUSTER HANDBOOK

Cluster Handbook

dealing

CLUSTER	dealing
AUDIENCE	Platform & on-call engineers
CLASSIFICATION	Internal — keep current as the platform evolves

Contents

> Plain-English guide to every component on this cluster. For each one: **Who** uses or maintains it, **What** it does, **How** it works in 30 seconds, and **Why** we chose it over alternatives.
>
> Audience: DevOps team. Assumes you know Kubernetes basics (Pod, Service, Deployment, NetworkPolicy) but not necessarily the specific tools we run.

Table of contents

1. The shape of the cluster

Before any of the named components, here's the physical picture.

- The cluster runs on **6 virtual machines** in your Proxmox cluster. All on the same VLAN (`10.10.120.0/24`).
- **3 are control-plane ("master") nodes**: `dealing-m-1/2/3` at `10.10.120.131-133`.
- **3 are worker nodes**: `dealing-w-1/2/3` at `10.10.120.134-136`.
- The control plane is reached through a single floating IP — `10.10.120.138` — that automatically moves to whichever master is currently the leader.
- External users reach apps via another floating IP — `10.10.120.140` — that the ingress controller owns.
- Each VM has two disks: a 50 GB OS disk and a smaller dedicated etcd disk (20 GB on masters, or a 20 GB data disk on workers).
- All VM-to-VM pod traffic across hosts is encrypted with WireGuard. App-to-app traffic on the same host is not (no need; it never leaves the host).
- Three off-cluster services are central, shared by every cluster your org runs: **Mimir** (metrics), **Loki** (logs), **Grafana** (dashboards + alerts). All at `*.stackflow.org`.

When you hear "the cluster," that's the 6 VMs. When you hear "the observability stack," some of it runs on-cluster (Prometheus, Alloy) and the storage/UI sits off-cluster.

---

2. Proxmox

- **Who:** The infrastructure team that owns the physical hardware.
- **What:** Open-source virtualization platform. Runs the 6 VMs that make up the dealing cluster.
- **How:** Proxmox combines KVM (the Linux kernel hypervisor) with a web UI and a CLI for managing VMs, storage, and networking. We tell Terraform what VMs to create; Terraform calls Proxmox's API; Proxmox spawns the VMs on top of the underlying hardware.
- **Why:** Open-source, no per-CPU license cost, mature, runs anywhere you can install Linux. Strong enough for production. Alternatives (vSphere, Hyper-V, OpenStack) would either cost real money or be much heavier to operate.

---

3. RKE2

- **Who:** Anyone running `kubectl` is using it indirectly. The DevOps team owns the upgrades.
- **What:** A Kubernetes distribution from Rancher/SUSE. Specifically, **RKE2 = "Rancher Kubernetes Engine 2"**.

- **How:** RKE2 is shipped as a single binary that, when started as a systemd unit, downloads and runs the Kubernetes core components (apiserver, controller-manager, scheduler, etcd) as **static pods** managed by kubelet. It also auto-deploys some bundled add-ons (Cilium-as-CNI, CoreDNS, metrics-server, snapshot-controller) using a built-in Helm controller that watches files in `/var/lib/rancher/rke2/server/manifests/`.
- **Why:** Faster setup than vanilla kubeadm, secure defaults (CIS-benchmark compliant, FIPS-friendly), runs on bare metal or VMs equally well, free, and from the same vendor as Rancher (so the upgrade path stays predictable). The alternative — kubeadm — works but you'd have to wire up everything RKE2 does for free yourself.

---

4. etcd

- **Who:** The Kubernetes control plane uses it. As a human, you should rarely touch it — but when it gets slow, the whole cluster gets slow, so you should know it's there.
- **What:** A small, strongly-consistent database that stores every single Kubernetes object (every Pod definition, every Service, every Secret, every NetworkPolicy). It runs as a 3-node cluster (one on each master) using the Raft consensus algorithm.
- **How:** Every write to the Kubernetes API ultimately ends up as a write to etcd. The apiserver is just an HTTPS wrapper with auth and validation; the data lives in etcd. The 3 etcd instances elect a leader; the leader accepts writes and replicates them to the others. As long as 2 of the 3 are alive, the cluster is fine.
- **Why:** Required by Kubernetes — apiserver only knows how to talk to etcd (in standard upstream; some distros like k3s have alternatives). The interesting choices we made:
- **Dedicated disk per master** (the 20 GB `scsi1` we discussed) so etcd's fsync I/O doesn't fight with anything else for disk time.
- **8 GB quota** (`quota-backend-bytes`) — once etcd's on-disk size hits this, the cluster goes read-only. We size for headroom.
- **6h snapshots, 10 retained** — local snapshots automatically taken; restore from one if the cluster catches fire.
- **Performance-tuned heartbeat/election timeouts** — slightly more aggressive than defaults, suitable for our local network.

---

5. kube-vip

- **Who:** Mostly invisible. It just makes the cluster API reachable at one IP no matter which master you actually connect to.
- **What:** A small daemon that runs on every master node. The three instances elect a leader; whoever wins claims the floating IP `10.10.120.138` and answers traffic for it. If the leader dies or loses a network heartbeat, another instance takes the IP within seconds.

- **How:** Two parts: 1. **ARP-based VIP** (control plane): the three kube-vip instances talk among themselves via the kube-apiserver. The leader sends ARP packets advertising `10.10.120.138 → leader's MAC address` on the local L2. Other hosts on the VLAN update their ARP tables and route traffic to whoever is the current leader. 2. **Service load-balancer** mode (optional, not in use here — Cilium provides this instead).
- **Why:** We need a single, stable IP for the kubeconfig and worker `--server=…` URLs. The alternatives:
- **External load-balancer** (F5, HAProxy box) — needs hardware and ops, and is itself a SPOF or yet another HA system.
- **MetalLB** — works but doesn't do control-plane VIPs.
- **DNS round-robin** to all 3 masters — slow failover (DNS TTL) and many clients don't honor it.

kube-vip is just three small pods, zero external dependencies, and the failover is fast (~3 seconds).

---

6. Cilium

This is the biggest single piece. Read this section if you read no other.

- **Who:** Every pod's network traffic passes through Cilium. The DevOps team writes NetworkPolicies and CCNPs that Cilium enforces.
- **What:** Cilium replaces the Kubernetes CNI plugin **and** kube-proxy **and** the Ingress controller, all with one set of BPF programs that run inside the Linux kernel.

What does it actually do for the cluster?

6.1 Pod networking (the CNI part)

Every pod gets an IP from a `/24` block carved out for its node (e.g. `10.42.4.0/24` on `dealing-w-1`). When a pod sends a packet, the BPF program on its node decides where the packet goes: - If the destination IP is on the same node → BPF redirects it directly to the destination pod (no network hop). - If the destination IP is on another node → BPF wraps it in a **vxlan** tunnel and ships it to the other node, where another BPF program unwraps it and delivers to the destination pod. - Pod-to-pod traffic between nodes is also encrypted with WireGuard (so the vxlan packet rides inside a WG tunnel on the wire).

The **kube-proxy replacement** part means there's no iptables-based DNAT for Service IPs. Cilium's BPF maintains a small in-kernel map of Service IP → backend pod IP and rewrites destination addresses there. Much faster than iptables (which scales linearly with rule count).

6.2 L2 announcements

Cilium watches Kubernetes Services of type LoadBalancer that have a designated VIP. For each, **one Cilium agent (the leader for that VIP) sends ARP packets on the LAN saying "I'm the answer for this IP."** That's how `10.10.120.140` (the IngressController) becomes reachable from anywhere on `10.10.120.0/24`. Failover works the same way as kube-vip — if the leader Cilium agent fails, another takes over.

This replaces MetalLB. Same idea, but built into Cilium and uses the same identity/policy machinery for security.

6.3 IngressController + Gateway API

Standard Kubernetes Ingress resources (and the newer Gateway API resources) get picked up by Cilium and turned into Envoy configuration. **Envoy runs embedded inside each cilium-agent pod** — it's the L7 proxy that terminates TLS and routes incoming HTTPS by hostname (e.g. `argocd.dealing.internal`) to the right backend Service.

This is the path: browser → `10.10.120.140:443` → Cilium L2 LB → Envoy in cilium-agent → backend Service → backend pod.

6.4 WireGuard node-to-node encryption

`encryption.enabled: true, type: wireguard`. Every Cilium agent sets up a WireGuard interface (`cilium_wg0`) and establishes peers with every other agent. Pod-to-pod traffic across nodes goes through this WG mesh, encrypted with modern Noise-protocol cryptography.

What we explicitly do NOT do: **`nodeEncryption: false`** — encrypting pod-to-node-host-IP traffic with WG. We tried, it broke metrics scraping for reasons buried in the Cilium 1.18 datapath. Pod-to-pod (the security-meaningful traffic) is still encrypted; pod-to-host-IP traffic rides the vxlan tunnel un-WG'd (still on the private VLAN).

6.5 NetworkPolicy enforcement

Both vanilla Kubernetes `NetworkPolicy` and Cilium's richer `CiliumNetworkPolicy` / `CiliumClusterwideNetworkPolicy` get compiled into BPF programs. When a packet hits the BPF, it's checked against the policy in O(1) time per rule.

- **Who:** Application/security teams write policies. Cilium enforces.
- **Why Cilium policies over plain K8s:** Cilium can match by labels, identities, FQDNs, L7 (HTTP methods, paths, DNS queries), and reserved identities. Plain K8s NetworkPolicy is L3/L4-only and only IP-based.

Cilium summary

Why we chose Cilium over alternatives (Calico, Flannel, Weave, …):

- Replaces THREE other components (CNI + kube-proxy + Ingress controller), so we run fewer moving parts.
- BPF datapath is faster than iptables-based ones for non-trivial cluster sizes.
- Hubble (next section) gives flow-level observability nothing else matches.
- One vendor's docs and support, not three.

7. Hubble

- **Who:** SRE/DevOps debugging "why can't X talk to Y." Also feeds Grafana dashboards for network visibility.
- **What:** A network flow logger and metrics exporter built into Cilium. Every packet decision Cilium makes (forward / drop / encrypt) generates a flow event.
- **How:** Three pieces:
- `hubble-agent` — runs inside every cilium-agent pod (DaemonSet). Reads BPF events from the kernel ring buffer in real time.
- `hubble-relay` — one deployment that fans out queries to every hubble-agent and aggregates results. The single endpoint your Hubble UI or CLI talks to.
- `hubble-ui` — web UI that visualizes flows in real time. Reachable at `hubble.cluster.internal`.
- **Why:** When something networking-related breaks (a NetworkPolicy denies, a Service IP doesn't resolve, an Envoy 403s), `hubble observe --verdict DROPPED` tells you exactly which packet was dropped and why. It's the single highest-leverage debugging tool for this cluster. We've already used it three times this week.

Note: Hubble doesn't capture L7 (HTTP/DNS) content unless a CiliumNetworkPolicy with `http: []` or `dns: []` rules is in effect on the relevant pods. Cilium's IngressController traffic IS L7-parsed by Envoy and produces `hubble_http_*` metrics, but pod-to-pod HTTP between workloads is L4-only until we opt in.

---

8. CoreDNS

- **Who:** Every pod resolves DNS through CoreDNS. Almost always invisible.
- **What:** A Go-based DNS server. Runs as 2 replicas (Deployment) and is the default cluster DNS server (`10.43.0.10`).
- **How:** It has plugins for different DNS functions chained in a "Corefile." Ours:
- `hosts` block adds custom static entries (we add `mimir.stackflow.org → 10.10.103.203` here because external secrets weren't an option).
- `kubernetes` plugin: resolves `*.svc.cluster.local` names by querying the Kubernetes apiserver for matching Services.
- `forward . /etc/resolv.conf` plugin: anything else, forward to the node's upstream DNS.
- **Why:** Standard Kubernetes DNS implementation. Ships with RKE2 by default. Alternative would be `kube-dns` (older, slower).

---

9. cert-manager

- **Who:** Anyone declaring a `Certificate` or annotating an `Ingress` with `cert-manager.io/cluster-issuer: ...`. Largely automatic.
- **What:** The cert-issuance robot. You declare a `Certificate` resource (or rely on automatic issuance via Ingress annotations); cert-manager generates a CSR, talks to a CA (Let's Encrypt, an internal CA, etc.), gets a signed cert back, and stores it in a Kubernetes Secret.
- **How:** Three pods:
- `cert-manager` controller — does the work (talk to CA, store secret).
- `cert-manager-cainjector` — auto-injects CA bundles into webhooks/APIService objects that need them.
- `cert-manager-webhook` — validates Certificate/Issuer YAML at API admission time.

Issuers in this cluster: - `cluster-ca-issuer` (Ready) — internal CA, used for `*.dealing.internal` and `*.cluster.internal`. - `selfsigned-issuer` (Ready) — bootstrap CA. - `letsencrypt-prod` / `letsencrypt-staging` (Not Ready) — configured but ACME solver isn't fully wired up. Out of scope today. - **Why:** Renewal is automated, expiry monitoring is built in, and the same code works against private CAs and public Let's Encrypt. Doing this by hand for every Ingress would be a half-time job.

10. external-secrets

- **Who:** The team's plan was for apps to declare an `ExternalSecret` and have it auto-sync from a central secret backend (HashiCorp Vault, Azure Key Vault, AWS Secrets Manager, etc.).
- **What:** A controller that watches `ExternalSecret` resources, fetches the actual secret from a backend, and writes it as a Kubernetes `Secret`.
- **How:** You configure a `ClusterSecretStore` once (telling it where the backend lives and how to authenticate). Then every `ExternalSecret` references the store and names which secret to fetch.
- **Why:** Avoids committing secrets to Git. Lets the security team rotate secrets centrally without needing to redeploy every app.

Status on this cluster: Deployed, but **not configured**. There's no `ClusterSecretStore` defined, and no `ExternalSecret` resources exist. Currently three pods doing nothing. Either pick a backend or remove the install.

11. ArgoCD

- **Who:** Anyone deploying applications to the cluster. The team's deploy pattern.

- **What:** GitOps controller. You commit a Helm/Kustomize/plain-YAML app to a Git repo; you write an `Application` resource pointing at that repo path; ArgoCD pulls the manifests and `kubectl apply`s them to the cluster. It continuously checks Git and reconciles drift.
- **How:** Several pods working together:
- `argocd-application-controller` (StatefulSet) — the reconcile loop.
- `argocd-repo-server` — clones git repos and renders manifests.
- `argocd-server` — the web UI and API.
- `argocd-applicationset-controller` — generates multiple Applications from one template (good for multi-cluster).
- `argocd-notifications-controller` — sends events to Slack/Teams/webhook.
- `argocd-redis` — caches manifest renders.
- **Why:** GitOps means "the cluster state is whatever the Git repo says." Audit trail, rollback by `git revert`, no one ever `kubectl apply`-s by hand again. Argo specifically (vs. Flux) has a much better web UI for inspecting application state and diffing live vs. Git.

Status on this cluster: Argo is running and reachable at `argocd.dealing.internal`, but **no Applications are defined yet** — Argo is idle. Once you start deploying workloads, you'll register Applications.

Admin password (initial): in Secret `argocd/argocd-initial-admin-secret`. Currently `JVIA0NfztsCBPvGs`. Change it.

---

12. kube-prometheus-stack

The metrics-scraping side of observability lives inside the cluster.

- **Who:** SRE/DevOps. Your dashboards and alerts ultimately read from data scraped here.
- **What:** A Helm chart that bundles Prometheus, the Prometheus Operator, kube-state-metrics, and (optionally) Alertmanager + Grafana. We use the first three; we send alerts via Grafana instead of the bundled Alertmanager.
- **How:**
- **Prometheus Operator** is a controller that creates Prometheus pods based on a `Prometheus` CR you define. It watches `ServiceMonitor` and `PodMonitor` resources and tells Prometheus to scrape those endpoints. Effectively a templating system on top of the raw Prometheus config.
- **Prometheus** (one StatefulSet pod, `prometheus-kube-prometheus-stack-prometheus-0`) pulls metrics every 30 seconds from every endpoint covered by a `ServiceMonitor`. It stores them locally for ~2 hours (for fast queries during incidents) and `remote-writes` them to **Mimir** for long-term storage. Mimir is the single source of truth; the local Prometheus is just a buffer.
- **kube-state-metrics (KSM)** is a separate process that talks to the Kubernetes API and emits one Prometheus metric per K8s object property. `kube_node_status_condition`, `kube_pod_container_status_restarts_total`, etc. — these all come from KSM, not from the kubelet.
- **Why:**

- Prometheus is the de-facto standard. Everything that emits metrics speaks the Prometheus exposition format.
- The Operator removes the need to hand-edit Prometheus configs; the `ServiceMonitor` model is much more Kubernetes-native.
- KSM is the only good way to get object-state metrics into Prometheus (the apiserver itself doesn't expose them as metrics).

---

13. Grafana Alloy

- **Who:** Runs as a systemd unit on every cluster node. You don't interact with it day-to-day; it just runs.
- **What:** A combined metrics exporter + log forwarder. Replaces what would otherwise be **two separate processes**: node_exporter (for host metrics) and Promtail (for log shipping).
- **How:** Alloy uses a config language (called Alloy syntax — it's River/HCL-ish). Our config does three things on each node: 1. `prometheus.exporter.unix` — exposes node-level metrics (CPU, RAM, disk, network, processes) on a local port. 2. `prometheus.scrape "node_metrics"` — scrapes that local port every 15s and `remote-writes` the results to Mimir. 3. `loki.source.file` — tails `/var/log/pods/**/*.log`, parses CRI log format, and ships each log line to Loki.

Both writes go to `mimir.stackflow.org` / `loki.stackflow.org` and carry the label `cluster_name=dealing` so we can filter to this cluster in Grafana.

- **Why:**
- Running as systemd (not as a Pod) means it can read `/sys/`, `/proc/`, and `/var/log/pods/` natively without privileged container shenanigans.
- Single binary = one less DaemonSet to manage. The chart's `nodeExporter` and Promtail are both disabled because Alloy covers both.
- Alloy is from the same vendor (Grafana Labs) as the rest of our observability stack — Mimir, Loki, Grafana — so the integration is well-supported.

---

14. Mimir

- **Who:** Where every metric ultimately lives. Queried by Grafana.
- **What:** A horizontally scalable Prometheus-compatible long-term metrics store. Off-cluster — runs at `mimir.stackflow.org` and is shared across all your org's clusters (10 of them today).
- **How:** Mimir accepts Prometheus's `remote-write` protocol over HTTP. Each cluster's Prometheus + Alloy push metrics in; Mimir spreads them across object storage (S3/GCS/disk) with millisecond-level query response thanks to a tiered query engine. Multi-tenancy is per-cluster via the `X-Scope-OrgID` HTTP header (we send `dealing`).
- **Why:**

- Single-replica Prometheus can't store more than a few weeks of data; Mimir keeps months/years.
- Sharing one Mimir across clusters means one Grafana, one set of dashboards.
- Open-source (AGPL'd), no vendor lock-in.

Alternatives — Thanos (similar architecture, slightly older), Cortex (predecessor), VictoriaMetrics (different ecosystem). Mimir's the most actively developed of the bunch.

---

15. Loki

- **Who:** Same role as Mimir, but for logs.
- **What:** A horizontally scalable, Prometheus-style log store. Off-cluster at `loki.stackflow.org`.
- **How:** Logs come in tagged with labels (just like Prometheus metrics). The labels are indexed; the log bodies are stored in object storage. You query with **LogQL** which looks like PromQL but operates on log lines. Alloy on each node ships logs there.
- **Why:** Cheap log storage (you only pay for object storage), tag-based not full-text-index-based (so it scales without breaking the bank like ELK does), and same labels/auth model as Mimir so one mental model.

---

16. Grafana

- **Who:** Where humans look at dashboards and where alerts fire from. Off-cluster.
- **What:** The visualization layer over Mimir, Loki, and other data sources. Also the alerting engine you've chosen.
- **How:** Grafana has data-source connections to Mimir (metrics) and Loki (logs). Dashboards (we have 5 in the `new-kube-cluster` folder) render queries against those sources. **Grafana managed alerts** evaluate PromQL queries on a schedule and fire alerts to **contact points** — for you, a Microsoft Teams webhook.
- **Why:** Universal interface to multiple backends; team can switch the storage layer underneath without changing dashboards. Alerting in Grafana (rather than Prometheus's Alertmanager) means one place to define + route alerts.

---

17. Microsoft Teams

- **Who:** Your on-call destination.
- **What:** Where alerts arrive.
- **How:** Grafana posts JSON to a Teams webhook URL. Teams renders it as a card in the configured channel.

- **Why:** Your org uses Teams. The integration is straightforward (caveat: Microsoft is deprecating the legacy "Incoming Webhook" connector in 2025; the replacement is "Workflows" — same general idea, different URL format).

---

18. Network policy (K8s + Cilium)

- **Who:** Platform team owns the cluster-wide CCNPs. App teams own per-app CNPs in their own namespaces. Network team owns VLAN ACLs at the switch.
- **What:** A 4-layer security model. Each layer has a specific job; no layer tries to duplicate another's.

``` Layer 1 — Switch VLAN ACLs (network team) VLAN-to-VLAN segmentation. Explicit cross-VLAN port allowlist. This is the network-layer security boundary.

Layer 2 — Cluster-wide CCNPs (platform team) 2 additive policies in `security/network-policies/`: • platform-baseline-egress — every pod ↔ in-cluster + DNS + apiserver • platform-baseline-ingress — IngressController/Prometheus/kube-system → pods Both use `enableDefaultDeny: false` so they're additive (allow-only, never force default-deny on the pods they select).

Layer 3 — Per-app CNPs (app team, optional) When an app needs tighter than baseline (e.g. payments → only stripe.com), the app team adds a CiliumNetworkPolicy in their namespace with `enableDefaultDeny.egress: true` and a narrow allowlist.

Layer 4 — App-layer TLS + authn (developers) mTLS, JWT, OAuth. Defense in depth on top of network policy. ```

- **How:** Cilium agents compile every NetworkPolicy / CNP / CCNP into BPF programs in the kernel. Each packet at egress/ingress is checked in O(1) per rule. Cilium identifies the source and destination by *identity* (a hash of pod labels) — including reserved identities like `host`, `world`, `kube-apiserver`, `ingress`, `remote-node`. K8s NetworkPolicy `ipBlock` rules only match the `cidr` reserved identity, so K8s NetworkPolicy can't restrict cross-node-IP traffic — for that you need a CNP with `toEntities`.
- **Why this shape:**
- **Switch ACLs already do segmentation.** Layering CIDR-blocks at the cluster level would either duplicate the switch (no value) or contradict it (broken cross-VLAN traffic). We tried it, broke things, undid it.
- **The cluster-wide CCNPs are intentionally permissive.** They grant common platform access (DNS, apiserver, in-cluster pod-to-pod) without forcing default-deny. This lets the DevOps team spin up new namespaces (`webtrader`, `crm`, `affiliate`, …) without writing any netpols.
- **Per-app tightening is opt-in, app-team-owned.** Each app decides if it needs tighter than baseline. No central bottleneck for new policy additions.
- **Resource types in play:**

| Resource | Scope | Capability |
|---|---|---|
| `networking.k8s.io/v1 NetworkPolicy` | Namespace | L3/L4 only (IP, port). Doesn't match Cilium reserved identities. |
```

Resource	Scope	Capability
`cilium.io/v2 CiliumNetworkPolicy` (CNP)	Namespace	L3/L4/L7 (HTTP method/path, DNS query patterns). Matches identities.
`cilium.io/v2 CiliumClusterwideNetworkPolicy` (CCNP)	Cluster	Same as CNP but global.

- **The footgun to avoid:** an empty selector (`{}`) matches every endpoint including reserved identities (`ingress` , `host` , `world` , `kube-apiserver`). Use `matchExpressions: [{key: k8s:io.kubernetes.pod.namespace, operator: Exists}]` to match all real pods but skip reserved identities. Kyverno now blocks `{}` in CCNPs (`policy-reviewer.md`).

19. PodSecurityStandards (PSS)

- **Who:** Kubernetes admission controller. Activated by labels on namespaces.
- **What:** Three pre-defined security profiles for what a Pod is allowed to declare in its `spec` . Replaces the deprecated PodSecurityPolicy (PSP).
- **How:** You label a namespace with `pod-security.kubernetes.io/enforce: <profile>` (or `audit` / `warn`). Profiles:

Profile	Permits
`privileged`	Everything. No restrictions. Use for system namespaces only.
`baseline`	Disallows obvious privilege escalation: hostPID, hostNetwork, hostPath (most uses), privileged containers, dangerous capabilities.
`restricted`	Hardened: only runs as non-root, must drop all caps, seccomp required, no host-* anything.

Current state on dealing: - `production` → `restricted` - `staging` , `development` , `argocd` → `baseline` - `cert-manager` → `restricted` - `monitoring` → `privileged` (Prometheus needs hostPath) - `external-secrets` , `kube-system` , `default` → no label

- **Why:** Cheap, builtin, no extra controller. Apply once, get hard guarantees forever.

20. Kyverno

- **Who:** All admission requests pass through Kyverno's webhook. As a human, you write ClusterPolicies.
- **What:** Policy-as-code engine for Kubernetes. Validates, mutates, or generates resources at admission time based on rules you write in YAML.

- **How:** Same admission-webhook pattern as cert-manager and the Prometheus operator. The apiserver POSTs every incoming resource to Kyverno's webhook; Kyverno evaluates each `ClusterPolicy` whose `match` block applies; if any policy says reject, the request fails.

Our current single policy (`ccnp-no-empty-endpoint-selector`) blocks CCNPs with `endpointSelector: {}` — the specific bug that broke Ingress on 2026-05-20.

- **Why:**
- Rules become code that runs every time. Humans forget; webhooks don't.
- YAML syntax, no Rego (unlike OPA/Gatekeeper, the main alternative). Lower learning curve.
- Can also do mutating (inject default labels, set default resources) and generating (auto-create NetworkPolicies for new namespaces). We start narrow but the surface is broad.

Cost trade-off to be aware of: Kyverno is in the write path for every API call. If it's down with `failurePolicy: Fail` (default), no one can apply YAML. With `Ignore`, policies stop being enforced silently. We run 2 admission-controller replicas and check it's alive via Kyverno's own metrics.

21. RKE2 bundled add-ons

Smaller pieces RKE2 ships and we leave at defaults.

Add-on	What it does	Why
CoreDNS	Cluster DNS (covered in §8)	Standard.
rke2-metrics-server	Powers `kubectl top` and HPA resource scaling	Required by HPA. Cheap.
rke2-snapshot-controller	Watches `VolumeSnapshot` CRs and triggers CSI snapshot calls	Future-proofing for stateful workloads. Currently no PVCs exist, so it sits idle.
rke2-runtimeclasses	Pre-defines `RuntimeClass` resources for non-default runtimes (gVisor, etc.)	Unused today.
rke2-ingress-nginx	(Disabled — Cilium IngressController replaces it)	We picked Cilium's instead.

22. Terraform & the IaC layout

The whole cluster is defined as code under `terraform/` and `clusters/`. High level:

```
terraform/
  proxmox/                    # 6 VMs definition (modules: master_node, worker_node)
    modules/master_node/      # one VM definition per master, with 2 disks
    modules/worker_node/      # one VM definition per worker, with 2 disks
    templates/                # cloud-init template files
  observability/              # kube-prometheus-stack + alerts deployment
    modules/prometheus/       # the Helm release wrapped in TF
  argocd/                     # ArgoCD install (Helm)
  (per cluster: additional terraform code for cert-manager, external-secrets, etc.)

clusters/
  dealing/                    # per-cluster vars + state
    proxmox.tfvars            # disk sizes, VM counts, network ranges
    observability.tfvars      # cluster-name, Mimir creds, alert webhooks
    argocd.tfvars             # ArgoCD bootstrap config
    kubeconfig.yaml           # admin kubeconfig (writeable only by ops)
    tfstate/                  # terraform state

rke2/
  configs/                    # rendered per-master/worker RKE2 configs (gitignored)
  scripts/                    # install scripts (install-master.sh, install-worker.sh,
install-alloy.sh)
  configs/audit-policy.yaml   # apiserver audit log policy

security/
  network-policies/           # K8s NetworkPolicy + Cilium CCNP YAMLs
  kyverno-policies/           # ClusterPolicy YAMLs

docs/
  cluster-handbook-dealing.md   # this doc
  workload-catalog-dealing.md   # what's deployed
  runbook-dealing.md            # incident response
  policy-reviewer.md            # policy checklist
  multi-cluster-operations-runbook.md  # fleet-level ops
```

Why this layout:

- Separates cluster-shape (`terraform/proxmox/`) from cluster-content (`terraform/observability/` , `terraform/argocd/`).
- Per-cluster variables in `clusters/<name>/` so adding a new cluster is a copy of `_template/` plus filling in IPs.
- Static security policies in `security/` so they can be diffed across clusters.

23. Pritunl

- **Who:** The team's VPN entry point. You connect through it to reach internal hostnames like `argocd.dealing.internal` .
- **What:** OpenVPN-compatible VPN server. Provides per-user authenticated access into the internal network.
- **How:** Out of scope of the cluster itself — runs on an external host (`10.10.16.82`). Issuing routes that include `10.10.120.0/24` so VPN clients can reach the cluster.

- **Why:** Standard self-hosted VPN. Audit log of who connected when. See `pritunl-jumpserver-audit-10.10.16.82.md` for the access path notes.

---

24. Port and listener inventory

The "what is this port and who owns it" reference. Use this when you see a port in a connection log, a NetworkPolicy decision, or `ss -tlnp` output.

24.1 Listening on node IPs (all 6 nodes, masters + workers — except where noted)

Port	Proto	Listener	Bind	Purpose	Notes
22	TCP	sshd	`0.0.0.0`	SSH	Standard. Access via Pritunl VPN.
53	UDP/TCP	systemd-resolved	`127.0.0.53`	Host DNS stub	Not the cluster DNS — that's CoreDNS on `10.43.0.10`.
111	UDP/TCP	rpcbind	`0.0.0.0`	NFS portmap	Default Ubuntu service; not actively used by the cluster.
323	UDP	chronyd	`127.0.0.1`	NTP client	Local time sync.
2379	TCP	etcd	node IP	etcd **client port** (mTLS)	**Masters only.** apiserver and other clients connect here.
2380	TCP	etcd	node IP	etcd **peer port** (mTLS)	**Masters only.** etcd members talk to each other on this.
2381	TCP	etcd	node IP + `127.0.0.1`	etcd **metrics** (HTTP)	**Masters only.** Read-only Prometheus exposition. Enabled by `etcd-expose-metrics: true`.
2382	TCP	etcd	`127.0.0.1`	etcd debug	Loopback only. Internal.
2112	TCP	kube-vip	`*`	kube-vip metrics	**Masters only.** Prometheus exposition.
4240	TCP	cilium-agent	node IP	Cilium **health-check endpoint**	Used by Hubble's reachability checks across nodes.
4244	TCP	cilium-agent	`*`	Hubble peer-service (gRPC)	hubble-relay connects to each agent's 4244 to stream flows.
6443	TCP	kube-apiserver	`*`	**Kube API server** (HTTPS, mTLS)	**Masters only.** Workers connect to `127.0.0.1:6443` via the local rke2-agent reverse proxy, which forwards to a real master through the VIP.

Port	Proto	Listener	Bind	Purpose	Notes
6443	TCP	rke2 (workers)	`127.0.0.1`	Local apiserver proxy	**Workers only.** Workers don't run apiserver; this is the rke2-agent's local proxy.
6444	TCP	rke2 (workers)	`127.0.0.1`	RKE2 internal	Workers only.
8472	UDP	(kernel vxlan)	`0.0.0.0`	**Cilium vxlan overlay**	All pod-to-pod traffic across nodes goes through this.
9345	TCP	rke2	`*`	RKE2 **supervisor / agent registration**	**Masters only.** New workers contact this on join.
9879	TCP	cilium-agent	`127.0.0.1`	Cilium gops debug	Loopback only. `go tool pprof`-friendly endpoint.
9890	TCP	cilium-agent	`127.0.0.1`	Cilium agent debug	Loopback only.
9962	TCP	cilium-agent	`*`	**Cilium agent Prometheus metrics**	Scraped by ServiceMonitor `cilium-agent`.
9963	TCP	cilium-operator	(operator pod IP)	**Cilium operator Prometheus metrics**	Scraped by ServiceMonitor; only on whichever 2 nodes run cilium-operator.
9964	TCP	cilium-envoy (embedded)	`0.0.0.0`	**Cilium Envoy admin metrics**	Scraped by ServiceMonitor `cilium-envoy` (added 2026-05-21).
9965	TCP	cilium-agent	`*`	**Hubble metrics** (flow exporter)	Scraped by ServiceMonitor `hubble`.
10010	TCP	containerd	`127.0.0.1`	Containerd metrics/CRI	Loopback only.
10248	TCP	kubelet	`127.0.0.1`	kubelet **healthz**	Loopback only.
10250	TCP	kubelet	`*`	**kubelet API + metrics** (HTTPS)	Scraped by ServiceMonitor `kubelet`. Also where apiserver does `kubectl exec/logs/port-forward`.
10257	TCP	kube-controller-manager	`*`	**KCM metrics** (HTTPS)	**Masters only.** Bind changed from `127.0.0.1` → `0.0.0.0` on 2026-05-20 so Prometheus can scrape.
10258	TCP	cloud-controller-manager	`127.0.0.1`	CCM internal	Loopback only. No cloud provider configured, so CCM is mostly idle.

Port	Proto	Listener	Bind	Purpose	Notes
10259	TCP	kube-scheduler	`*`	**Scheduler metrics** (HTTPS)	**Masters only.** Same fix as KCM on 2026-05-20.
12345	TCP	alloy	`127.0.0.1`	**Grafana Alloy UI / metrics**	Loopback only. Hit via SSH+port-forward.
51871	UDP	(kernel WireGuard)	`0.0.0.0`	**Cilium WireGuard**	Encrypted pod-to-pod transport across nodes.

24.2 LoadBalancer / external IPs (Cilium L2 announcements)

IP	Port(s)	Service	Backed by	Purpose
`10.10.120.138`	6443	(control plane VIP)	kube-vip leader election	Kubeconfig and worker join URL. ARP'd by whichever master is currently kube-vip leader.
`10.10.120.140`	80, 443	`kube-system/cilium-ingress`	Cilium IngressController	All Ingress traffic (argocd.dealing.internal, hubble.cluster.internal).

24.3 Cluster-internal Service IPs (10.43.0.0/16)

A few key ones — the rest follow the normal `kubectl get svc -A` pattern:

Service	ClusterIP	Port(s)	Used by
`default/kubernetes`	`10.43.0.1`	443	Pods talking to kube-apiserver — DNAT'd by Cilium kube-proxy replacement to the apiserver static pods
`kube-system/rke2-coredns-rke2-coredns`	`10.43.0.10`	53	All pod DNS lookups
`kube-system/hubble-peer`	`10.43.214.53`	443	hubble-relay → per-node hubble-agents (gRPC)
`monitoring/kube-prometheus-stack-prometheus`	`10.43.222.102`	9090	Prometheus query API; remote-write source
`monitoring/kube-prometheus-stack-kube-state-metrics`	`10.43.51.179`	8080	KSM scrape target
`argocd/argocd-server`	`10.43.172.88`	80, 443	Backend for `argocd.dealing.internal` Ingress
`kube-system/hubble-ui`	`10.43.230.136`	80	Backend for `hubble.cluster.internal` Ingress

24.4 Quick reference: "what's port X?"

Port	Mnemonic
2379/2380	etcd (client/peer)
2381	etcd metrics
4240/4244	Cilium agent health / Hubble peer
6443	apiserver
8472	Cilium vxlan
9345	RKE2 supervisor
9962/9963/9964/9965	cilium-agent / cilium-operator / cilium-envoy / hubble metrics
10250	kubelet
10257/10259	KCM / scheduler
51871	Cilium WireGuard

24.5 If you see a port you don't recognize

Quick diagnosis path on any node:

```
sudo ss -tlnp sport = :<port>    # who is listening?
sudo lsof -i :<port>             # alternate; might require lsof install
ps -p <pid> -o pid,user,cmd      # what process is it?
```

For a *connection* you don't recognize:

```
sudo ss -tnp state established '( dport = :<port> or sport = :<port> )'
```

For a *blocked* connection (the kind we've been chasing this week):

```
AGENT=$(kubectl -n kube-system get pod -l k8s-app=cilium --field-selector spec.nodeName=$
(hostname) -o jsonpath='{.items[0].metadata.name}')
kubectl -n kube-system exec $AGENT -c cilium-agent -- hubble observe --verdict DROPPED --
last 50 | grep ':<port>'
```

25. End-to-end flows — "what happens step by step"

This section walks through the most common traffic flows on this cluster. Use it to explain to someone else how the cluster actually works. Each flow: - Starts with the user's intent - Lists every hop, with the source IP, destination IP, port, and the component doing the work - Ends with "what happens" in plain English

25.1 An external user opens `https://argocd.dealing.internal`

What the user wants: the ArgoCD UI in their browser.

```
1. Browser ─DNS─▶ resolver returns 10.10.120.140
   • The user is on the corporate network. Internal DNS (or their /etc/hosts) maps
     argocd.dealing.internal → 10.10.120.140.

2. Browser ─TCP/443─▶ 10.10.120.140
   • 10.10.120.140 is the Cilium IngressController LB IP.
   • Cilium's L2 announcements have ARP'd this IP from one of the dealing nodes
     (leader election picks one cilium-agent — let's say dealing-w-1).
   • The corporate switch routes the TCP SYN to dealing-w-1.

3. dealing-w-1 NIC ─▶ Cilium-Envoy (embedded in cilium-agent pod, port 9964 is its
                                    admin port; 80/443 is the IngressController listener)
   • Envoy accepts the TLS handshake and terminates TLS using argocd-server-tls
     (issued by cluster-ca-issuer in cert-manager).

4. Envoy reads the HTTP Host header = "argocd.dealing.internal"
   • Looks up its compiled config (built from the Ingress resource in argocd ns).
   • Finds: route → argocd-server.argocd.svc.cluster.local on port 80.

5. Envoy ─HTTP/80─▶ Service IP 10.43.172.88 (argocd-server)
   • Cilium kube-proxy-replacement (BPF) translates the Service IP to one of the
     argocd-server pod IPs (e.g. 10.42.4.X on dealing-w-1).
   • Pod traffic uses WireGuard encryption between nodes (here, same node so no WG).

6. argocd-server pod (port 8080 inside the container) serves the HTML/JS.

7. Response retraces steps 5 → 4 → 3 → 2 → 1.
```

Key ports: 443 (external HTTPS), 80 (Envoy → backend), 8080 (argocd-server container).

25.2 A pod calls an external API (e.g., `https://api.github.com`)

```
1. App pod (10.42.4.x) —DNS query—▶ CoreDNS (10.43.0.10:53 UDP)
   • DNS is allowed by CCNP-1 (platform-baseline-egress).

2. CoreDNS —?—▶ resolves
   • If hostname matches CoreDNS's `hosts` plugin (e.g. mimir.stackflow.org), return
locally.
   • Otherwise, forward via `forward . /etc/resolv.conf` — which inside the CoreDNS pod
points
     to the node's resolv.conf — which points to 8.8.8.8 (Google DNS).
   • CoreDNS sends the upstream query out (8.8.8.8:53). Switch ACL allows public DNS.
   • Reply comes back: api.github.com → 140.82.x.x.

3. App pod —TCP/443—▶ 140.82.x.x (api.github.com)
   • This is "world" identity in Cilium. K8s default-allow lets it through; no CCNP blocks
it.
   • Cilium BPF on the node routes the packet to eth0 with NAT (source rewritten to node IP
     for the external trip).

4. Switch routes the packet out to the public internet via the cluster's gateway.

5. Response comes back via the same path; Cilium un-NATs on receive; delivers to the pod.
```

Key ports: 53/UDP (DNS), 443/TCP (HTTPS to external).

25.3 A pod calls Mimir at `10.10.103.203` (cross-VLAN internal)

Same start as 25.2 but the destination is a known internal IP. The path that matters:

```
1. App pod —DNS—▶ CoreDNS hosts plugin: mimir.stackflow.org → 10.10.103.203 (no upstream
lookup).
2. App pod —HTTP/80—▶ 10.10.103.203 (Mimir gateway)
3. Cilium BPF, no policy block (CCNP-1 covers nothing about CIDR; no DENY policies exist).
4. Switch fabric: the dealing VLAN (10.10.120.0/24) → observability VLAN (10.10.103.0/24).
   The switch ACL between these VLANs explicitly allows port 80.
5. Mimir gateway receives the remote_write payload.
```

Key port: 80/TCP (HTTP to Mimir's `/api/v1/push` endpoint with `X-Scope-OrgID: dealing`).

25.4 Prometheus (in-cluster) scrapes kubelet metrics on every node

```
1. Prometheus pod (10.42.4.11 on dealing-w-1) wakes every 30s.
   • Reads ServiceMonitor "kube-prometheus-stack-kubelet" → 6 scrape targets (one per node).

2. Prometheus pod —HTTPS/10250→ 10.10.120.131-136 (the 6 dealing nodes)
   • Source: 10.42.4.11 (pod IP). Destination: node IP on port 10250.
   • Cilium identifies destination as `host` (own node) or `remote-node` (other nodes).
   • CCNP-1 allows this via `toEntities: [host, remote-node]`.

3. Cilium WireGuard between nodes: pod traffic on the underlay is WG-encrypted.

4. kubelet on each node receives, authenticates via Bearer token from Prometheus's SA,
   responds with metrics from /metrics, /metrics/cadvisor, /metrics/probes.

5. Prometheus pod appends to local TSDB (~2h cache).

6. Prometheus pod —HTTP/80→ 10.10.103.203/api/v1/push (Mimir, every 30s of remote-write)
   • Same path as 25.3.
```

Key ports: 10250/TCP (kubelet HTTPS metrics), 80/TCP (remote-write to Mimir).

25.5 Alloy (systemd on each host) scrapes node-level metrics

```
1. Alloy on dealing-m-1 (running as systemd, NOT a pod):
   • `prometheus.exporter.unix` collects CPU, memory, disk, network from /proc, /sys.
   • Listens on 127.0.0.1:12345 internally.

2. Alloy —scrapes→ itself (loopback)
   • `prometheus.scrape "node_metrics"` reads from the local exporter every 15s.

3. Alloy —HTTP/80→ mimir.stackflow.org/api/v1/push
   • External labels include cluster_name=dealing, node=dealing-m-1, environment=production.
   • Switch routes cross-VLAN.

4. Mimir ingests. Tagged with cluster_name="dealing".
```

Key port: 80/TCP (Alloy → Mimir). Alloy never touches the Kubernetes network — it's fully outside the cluster, talks directly to upstream Mimir.

25.6 A pod logs something → it ends up in Loki

```
1. App pod writes to stdout/stderr.
2. containerd captures the streams and writes to /var/log/pods/<ns>_<pod>_<uid>/<ctr>/
<n>.log
   (the standard CRI log location).
3. Alloy on that node:
   • `loki.source.file` tails /var/log/pods/**/*.log.
   • `stage.cri` parses the CRI log format (extracts stream, flags, log fields).
   • `stage.regex` extracts namespace/pod/container from the file path.
4. Alloy —POST→ loki.stackflow.org/loki/api/v1/push
   • Labels added: cluster, node, namespace, pod, container, stream.
5. Loki ingests, indexes labels, stores log bodies in object storage.
6. You query in Grafana Explore → Loki: {namespace="webtrader"} |= "error".
```

Key port: typically 80/TCP (Loki HTTP push API).

25.7 An image pull (when a new pod is scheduled)

This is the path that does NOT go through Cilium policy — it's host-level by containerd, not by the pod.

```
1. Scheduler decides pod will run on dealing-w-1.
2. kubelet on dealing-w-1 reads pod spec → "image: ghcr.io/example/app:v1.2.3"
3. kubelet asks containerd to pull the image.
4. containerd on dealing-w-1:
   • Reads /etc/containerd/config.toml for registry mirrors / auth.
   • Resolves ghcr.io via the HOST's DNS resolver (not CoreDNS).
   • TLS handshake to ghcr.io:443.
5. Switch routes outbound to the internet.
6. containerd unpacks the image to /var/lib/rancher/rke2/agent/containerd/.
7. kubelet starts the container.
```

Key port: 443/TCP (host-level pull). **No Cilium policy applies** — this is host-network traffic, not pod-network.

25.8 A Grafana alert fires → reaches Microsoft Teams

```
1. Grafana (at 10.10.103.203:3000) evaluates alert rules every minute.
2. Alert rule queries Mimir (PromQL) — same machine, internal call.
3. Condition met (e.g. "Cluster CPU > 85% for 10m").
4. Grafana's contact point: Microsoft Teams Workflows webhook URL.
5. Grafana ─HTTPS/443─► <teams-workflow-webhook-url>
   • Posts a JSON payload.
6. Teams renders the alert as a card in the configured channel.
7. On-call human sees it, opens runbook-dealing.md or the dashboard the alert linked.
```

Key port: 443/TCP (Grafana → Teams Workflows webhook). Grafana lives outside the cluster so this never touches Cilium.

25.9 `kubectl apply` from an admin's laptop

```
1. Admin runs `kubectl get pods`.
2. kubectl reads ~/.kube/config → server URL is https://10.10.120.138:6443.
   • 10.10.120.138 is the kube-vip-owned control-plane VIP.

3. Laptop ─TCP/6443─► 10.10.120.138 (via VPN, Pritunl)
   • The switch routes to whichever dealing master currently holds the VIP.
   • kube-vip leader-elected daemon on that master answers ARP for 10.10.120.138.
   • The TCP packet hits kube-apiserver on that master.

4. kube-apiserver:
   • TLS handshake with client cert from kubeconfig.
   • Authenticates the user (system:masters group via cert CN).
   • Authorizes via RBAC.
   • For a GET: reads from its in-memory cache OR queries etcd at https://127.0.0.1:2379.
   • For a CREATE/UPDATE: validates, applies admission webhooks (Kyverno, cert-manager,
     prom-operator), then writes to etcd. etcd Raft replicates to the other 2 members
     (port 2380).
   • Returns response to the caller.

5. kubectl prints the response.
```

Key ports: 6443/TCP (kube-apiserver mTLS), 2379/TCP (etcd client), 2380/TCP (etcd peer).

25.10 A worker pod sends a packet to another worker pod (different node)

The case Cilium's data plane was built for.

```
1. Source pod (10.42.4.11 on dealing-w-1) —TCP/8080→ Destination pod (10.42.5.42 on
dealing-w-2)
2. Cilium BPF on dealing-w-1:
   • Looks up destination IP in the ipcache map → identity: dealing-w-2's pod identity.
   • Checks egress policy in the BPF policy map → allowed (CCNP-1, default-allow).
   • Routes packet via cilium_wg0 (WireGuard interface) — pod-to-pod traffic between nodes
     is encrypted.
3. WireGuard wraps the packet, ships out eth0 to dealing-w-2's eth0 (UDP/51871).
4. dealing-w-2 cilium_wg0 decrypts → packet appears with source 10.42.4.11, dest 10.42.5.42.
5. Cilium BPF on dealing-w-2:
   • Checks ingress policy on the destination → allowed (CCNP-3, ingress from kube-system
     and intra-cluster; CCNP-1 has no ingress rule but doesn't deny).
   • Delivers to the destination pod's veth.
6. Destination pod's kernel hands the TCP segment to the application listening on 8080.
```

Key ports: application port (varies — 8080 in example), 51871/UDP (WireGuard transport).

25.11 End-to-end flow for a new "webtrader" app (the full user path)

This brings it all together. Imagine DevOps deploys a webtrader app and a user hits it.

```
Step 1 — DevOps creates the namespace and deploys:
  $ kubectl create namespace webtrader
  $ kubectl -n webtrader apply -f webtrader-deployment.yaml  (via ArgoCD)
  • Ingress created with host webtrader.dealing.internal → Service webtrader-svc:80.
  • No NetworkPolicy needed — CCNP-1 + CCNP-3 grant baseline already.

Step 2 — Cert-manager issues TLS for webtrader.dealing.internal:
  Ingress annotation cert-manager.io/cluster-issuer: cluster-ca-issuer
  → cert-manager creates Certificate resource → contacts internal CA → secret
    webtrader-tls is created → Cilium IngressController picks it up.

Step 3 — User visits https://webtrader.dealing.internal:
  • Browser → DNS → 10.10.120.140 (Ingress LB IP)
  • Browser → TLS handshake → Cilium-Envoy on a dealing node
  • Envoy looks at Host header → routes to webtrader-svc Service IP
  • Cilium BPF translates Service IP → webtrader pod IP
  • Pod serves the HTML

Step 4 — Webtrader pod calls api.github.com (e.g. to fetch some data):
  • Pod → CoreDNS (10.43.0.10:53) → forwards to 8.8.8.8 → IP returned
  • Pod → 140.82.x.x:443 → switch ACL allows public egress
  • GitHub returns response

Step 5 — Webtrader pod calls an internal db on cross-VLAN (e.g. 10.10.114.5:5432):
  • Pod → 10.10.114.5:5432
  • Cilium has no block (CCNP-1 is allow-only, no deny anywhere)
  • Switch ACL: this cross-VLAN Postgres port is explicitly allowed → permitted.
  • Postgres responds.

Step 6 — Prometheus scrapes webtrader's /metrics:
  • Prometheus pod → webtrader pod IP on port 9090 (or whatever /metrics port)
  • CCNP-3 (ingress) allows monitoring/prometheus → all pods
  • webtrader returns its metrics
  • Prometheus → Mimir (Step 5 in 25.3 above)

Step 7 — Webtrader pod writes a log line:
  • stdout → containerd → /var/log/pods/...
  • Alloy tails → Loki cross-VLAN (10.10.103.x switch-permitted)

Step 8 — User sees the page; if anything goes wrong, on-call gets a Teams alert,
         opens Grafana, sees the spike on the Overview dashboard, drills into
         Hubble Drops if it's a network issue or container_oom_events if it's a workload
issue.
```

That's the entire stack on one page. Anyone you onboard can read this and understand the cluster.

Term	Meaning
CNI	Container Network Interface. The plugin model Kubernetes uses for pod networking. Cilium is our CNI.
CRD	Custom Resource Definition. Lets you define your own resource kinds (`Certificate`, `ArgoApplication`, …) that behave like first-class Kubernetes objects.
CCNP	CiliumClusterwideNetworkPolicy. Cilium-specific NetworkPolicy that works globally.
PSS	PodSecurityStandards. The successor to PSP (PodSecurityPolicy, deprecated).

Term	Meaning
Reserved identity	In Cilium, a fixed identity assigned to special endpoints: `host`, `world`, `remote-node`, `kube-apiserver`, `ingress`, `unmanaged`, `init`, `health`.
GitOps	Pattern where Git commits → cluster state. The cluster is reconciled to whatever Git says.
L7	Application layer. HTTP, DNS, gRPC — anything where the meaning is in the bytes, not just IP/port.
Remote-write	Prometheus protocol for pushing metrics to a long-term store like Mimir.
External labels	Labels that Prometheus/Alloy add to every metric on the way out — used to tag which cluster the metric came from.

This handbook is alive — when you onboard a new component or change how something works, update the corresponding section. The companion docs (catalog, runbook, policy-reviewer) cross-link back to here.